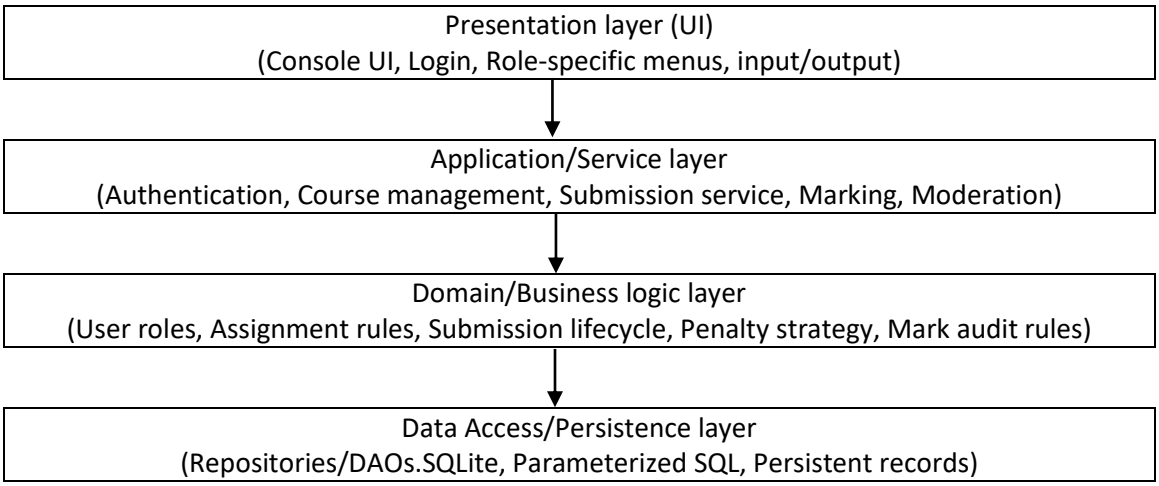
PHASE 3: SYSTEM DESIGN

1. **System Architecture**
    - **Architectural style selection**
      The Moderated Learning Management System (MLMS) adopts a Layered Architecture Pattern structured into four distinct, loosely coupled layers:

        - **Presentation/UI layer (console interface):** Handles user interaction, menu rendering, input capture and display of system outputs.
        - **Application/Service layer**: Enforces workflow rules, authorization checks, state transitions, mark moderation logic and business workflows.
        - **Domain/ Business logic layer**: Contains domain entities (User, Assignment, Submission, Mark) and encapsulates fundamental domain rules (e.g penalty rules, extension validity.
        - **Data access/Persistence layer:** Encapsulates SQLite interactions using parameterized queries to enforce data persistence and security.

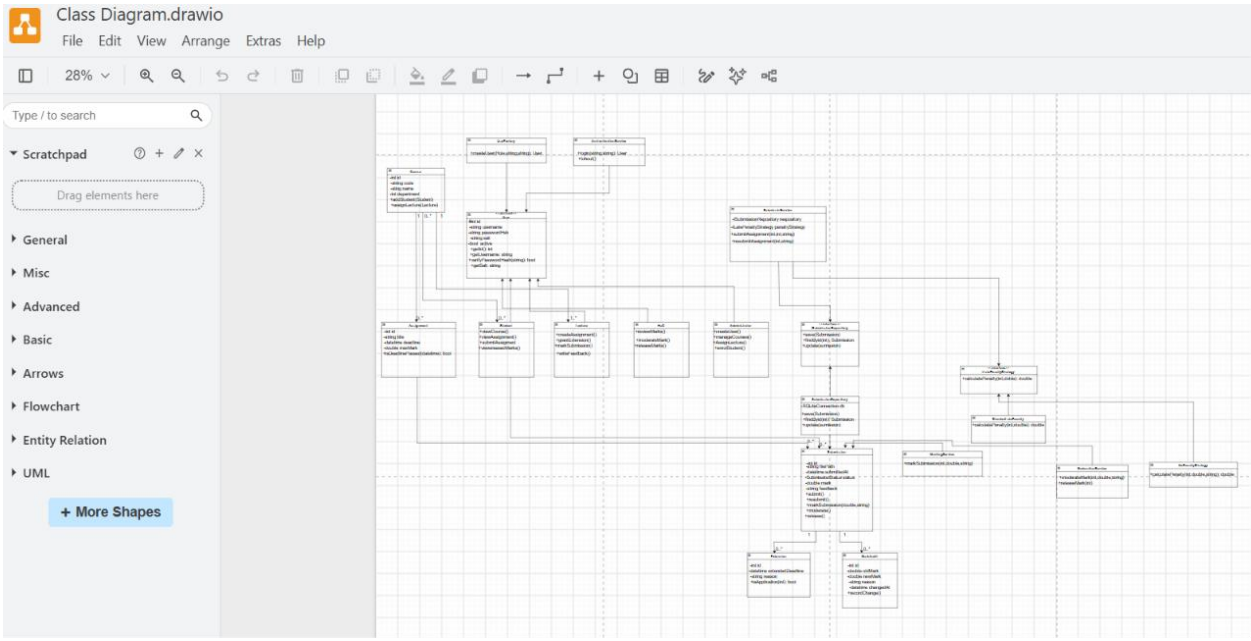


- **Architecture rationale**
    - **Separation of concerns:** The interface, business rules and database operations are kept separate, making the system easier to maintain.
    - **Security:** Authentication and role checks can be enforced in application services before protected operations are performed.
    - **Testability:** Business logic can be tested independently of the console interface and SQLite database.
    - **Maintainability:** Changes to interface or database can be made without rewriting the entire system.
    - **Office operation:** SQLite stores data locally, allowing the school to use the application without an internet connection.

- **Selected and rejected architectural style**
  - **Selected: Layered architecture**
    Provides clear separation of concerns, simplifies unit testing for individual business services, and directly supports maintaining an offline C++ desktop/console application.

  - **Rejected alternative: Model-View-Controller (MVC)**
    While standard for interactive GUI frameworks, pure MVC introduces excessive event binding complexity and UI framing boilerplate unnecessary for a desktop centric/console C++ application. Layered architecture provides clear separation between core business logic and storage while keeping UI logic straightforward.

- **Class diagram**



\+ public
\- private
\# protected
<<interface>> indicates an interface
1,0...1,*,1..* represent multiplicities

Design clarification: A submission is linked to an assignment and a student. An extension belongs to a particular student and assignment. The diagram represents the core domain; the database design below uses an Enrolment association entity to implement the many-to-many relationship between students and courses.

Pattern	Application	Justification
Strategy	ILatePenaltyStrategy StandardLatePenalty NopenaltyStrategy	Different penalty policies can be selected without

		changing submission logic.
Repository/DAO	ISubmissionRepository SubmissionRepository	Separates business logic from SQLite queries and enables testing with a substitute repository.
Factory	UserFactory	Centralizes the creation of Student, Lecturer, HoD and Administrator objects.

- **SOLID principles**

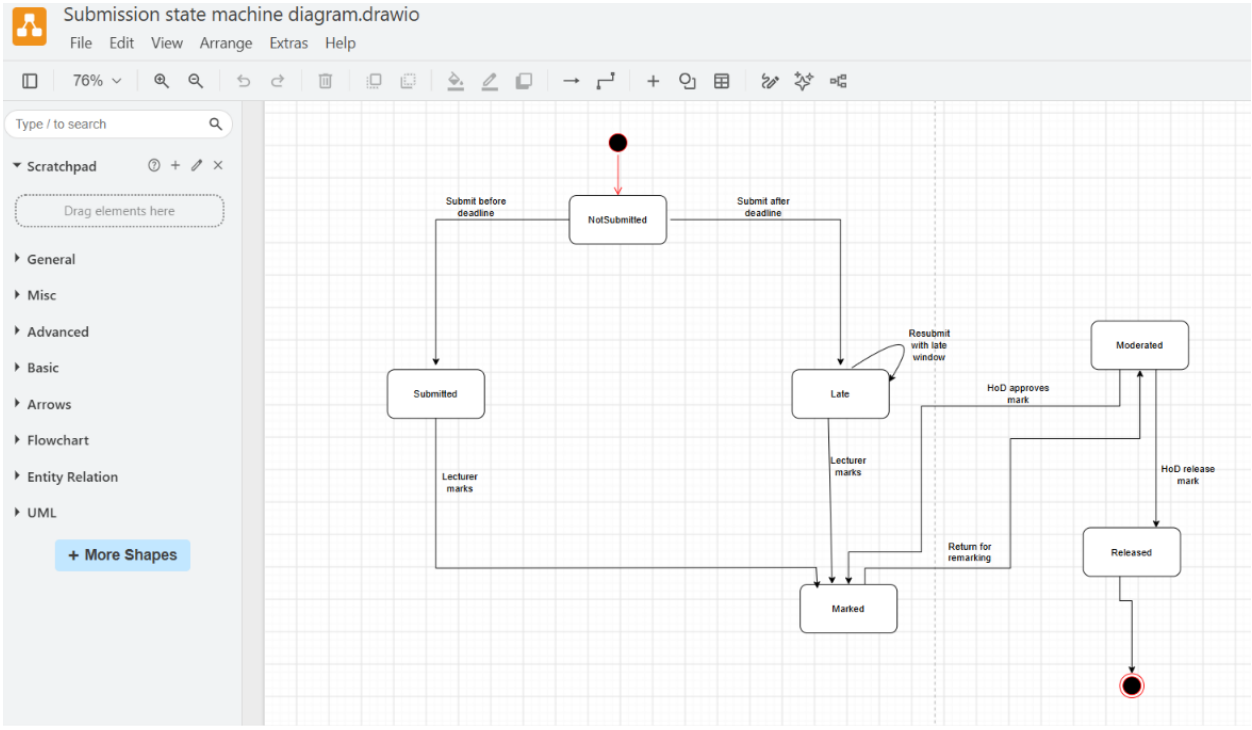
  - Single Responsibility Principle (SRP): Authentication, submission processing, marking and persistence are handled by different classes.

  - Open/Closed Principle (OCP): New penalty strategies can be added without modifying "**SubmissionService".**
  - Liskov Substitution Principle (LSP): Each user subclass can be used where a "**User"** is expected, while services still enforce role-specific permissions.
  - Interface Segregation Principle (ISP): Repository interfaces expose the operations required by their clients instead of one oversized interface.
  - Dependency Inversion Principle (DIP): Application services depend on repository and penalty interfaces rather than concrete SQLite or penalty implementations.

2. Behavioural models

The behavioural models describe how the LMS changes state and how its actors interact from submission until marks are released.

Submission state machine diagram

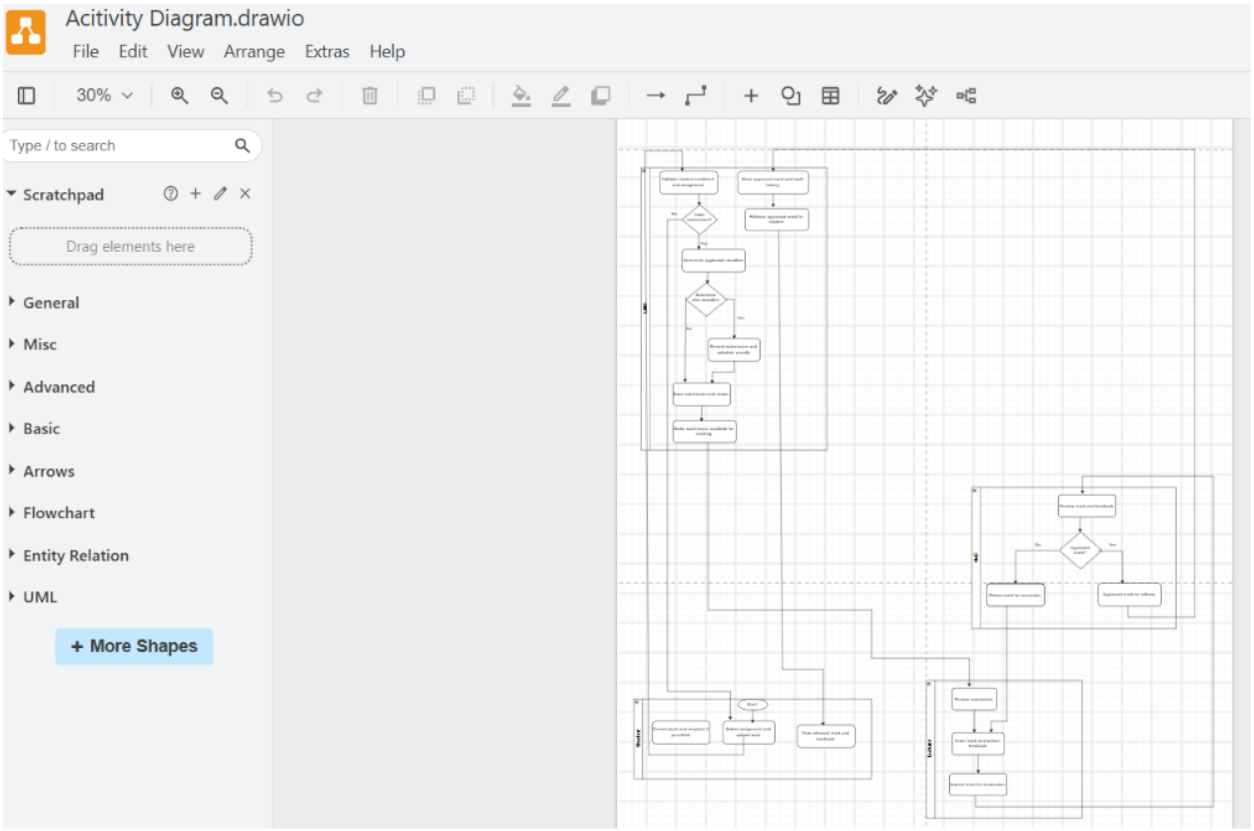


State descriptions:

State	Meaning
NotSubmitted	The student has not submitted work
Submitted	Work was submitted by the applicable deadline
Late	Work was submitted after the applicable deadline
Marked	The lecturer has recorded a mark and feedback
Moderated	The HoD has reviewd and approved the mark
Released	The approved mark and feedback are visible to the student

The application must store the submission timestamp and current state. Each state transition should be validated so, for example, a student cannot release a mark or a lecturer cannot bypass moderation.

Activity Diagram

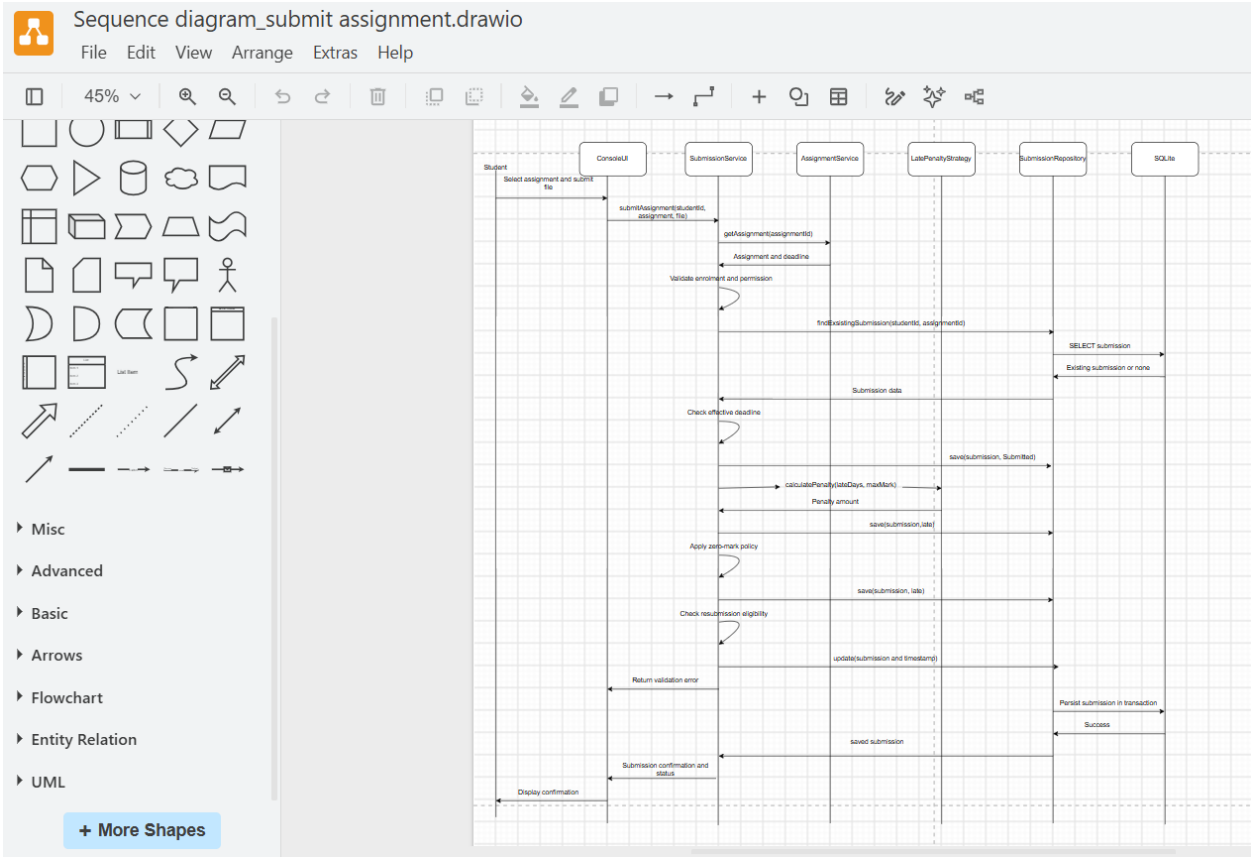


Process explanation

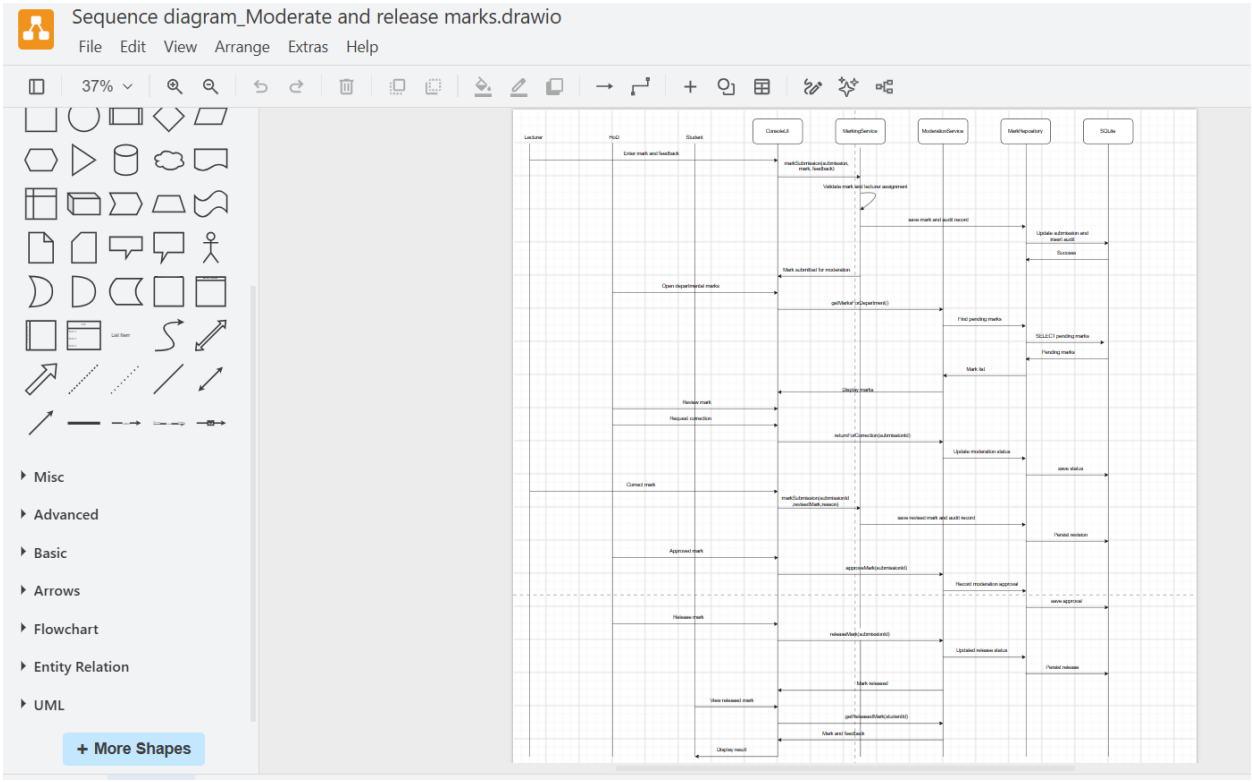
- The student selects an assignment and submits work.
- The LMS validates the student's enrolment and determines the effective deadline.
- The system classifies the work as on time or late and calculates the applicable penalty.
- The lecturer marks the work and supplies feedback.
- The HoD reviews the mark and either returns it for correction or approves it.
- Once approved, the HoD releases the result.

- The student can then view the mark and feedback.

Sequence diagram: Submit assignment



Sequence diagram: Moderate and release marks



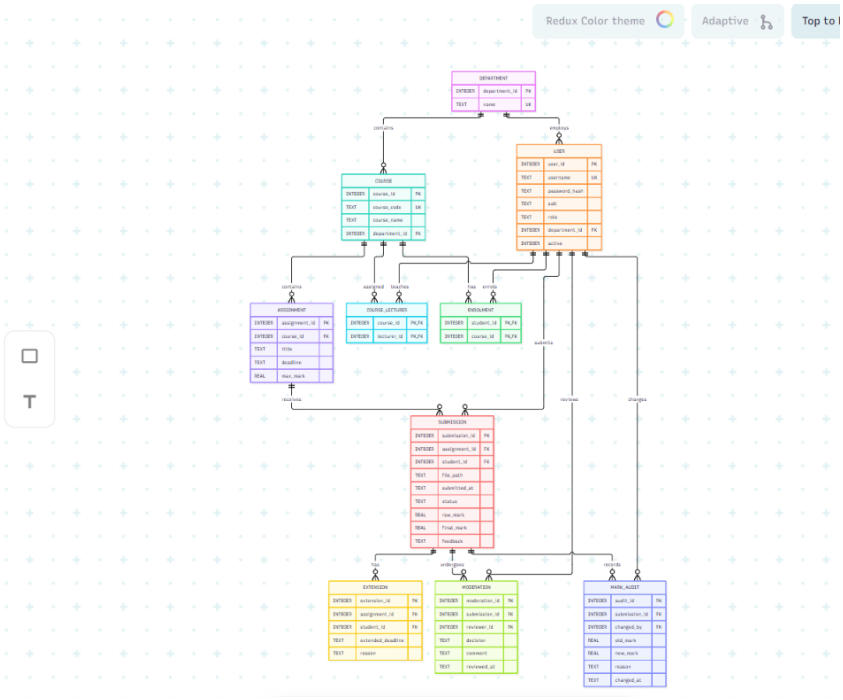
3. ERD

```
erDiagram
    DEPARTMENT ||--o{ COURSE : contains
    DEPARTMENT ||--o{ USER : employs
    USER ||--o{ COURSE_LECTURER : teaches
    COURSE ||--o{ COURSE_LECTURER : assigned
    USER ||--o{ ENROLMENT : enrols
    COURSE ||--o{ ENROLMENT : has
    COURSE ||--o{ ASSIGNMENT : contains
    ASSIGNMENT ||--o{ SUBMISSION : receives
    USER ||--o{ SUBMISSION : submits
    SUBMISSION ||--o{ EXTENSION : has
    SUBMISSION ||--o{ MARK_AUDIT : records
    USER ||--o{ MARK_AUDIT : changes
    SUBMISSION ||--o{ MODERATION : undergoes
    USER ||--o{ MODERATION : reviews

    DEPARTMENT {
        INTEGER department_id PK
        TEXT name UK
    }

    USER {
        INTEGER user_id PK
        TEXT username UK
        TEXT password_hash
        TEXT salt
        TEXT role
        INTEGER department_id FK
        INTEGER active
    }

    COURSE {
        INTEGER course_id PK
        TEXT course_code UK
        TEXT course_name
        INTEGER department_id FK
    }
```



Data dictionary

The following dictionary describes the main database fields. All timestamps use a consistent ISO 8601 representation.

Table	Field	Type	Constraint/meaning
DEPARTMENT	department_id	INTEGER	PK;unique department identifier
DEPARTMENT	name	TEXT	UNIQUE,NOT NULL
USER	user_id	INTEGER	PK
USER	username	TEXT	UNIQUE,NOT NULL
USER	password_hash	TEXT	NOT NULL;salted password hash
USER	salt	TEXT	NOT NULL;salt used for hashing
USER	role	TEXT	Student,Lecturer,HoD or Administrator
USER	department_id	INTEGER	FK;nullable where not applicable

USER	active	INTEGER	1 = active, 0 = disabled
COURSE	course_id	INTEGER	PK
COURSE	course_code	TEXT	UNIQUE. NOT NULL
COURSE	course_name	TEXT	NOT FULL
COURSE	department_id	INTEGER	FK to DEPARTMENT
ENROLMENT	student_id	INTEGER	PK,FK to USER
ENROLMENT	aourse_id	INTEGER	PK,FK to COURSE
ASSIGNMENT	assignment_id	INTEGER	PK
ASSIGNMENT	course_id	INTEGER	FK to COURSE
ASSIGNMENT	title	TEXT	NOT NULL
ASSIGNMENT	deadline	TEXT	NOT NULL; original deadline
ASSIGNMENT	max_mark	REAL	Positive maximum mark
SUBMISSION	submission_id	INTEGER	PK
SUBMISSION	assignment_id	INTEGER	FK to ASSIGNMENT
SUBMISSION	student_id	INTEGER	FK to USER
SUBMISSION	file_path	TEXT	Stored location of submitted work
SUBMISSION	submitted_id	TEXT	Submission timestamp
SUBMISSION	status	TEXT	Lifecycle status
SUBMISSION	raw_mark	REAL	Lecturer’s mark before penalty
SUBMISSION	final_mark	REAL	Mark after penalty
SUBMISSION	feedback	TEXT	Lecturer’s written feedback
EXTENSION	extension_id	INTEGER	PK
EXTENSION	assignment_id	INTEGER	FK to ASSIGNMENT
EXTENSION	extended_deadline	TEXT	Approved revised deadline
EXTENSION	reason	TEXT	Reason for extension
MARK_AUDIT	audit_id	INTEGER	PK
MARK_AUDIT	submission_id	INTEGER	FK to SUBMISSION
MARK_AUDIT	changed_by	INTEGER	FK to USER
MARK_AUDIT	old_mark	REAL	Previous mark
ARK_AUDIT	new_mark	REAL	Updated mark
MARK_AUDIT	reason	TEXT	Mandatory explanation
MARK_AUDIT	changed_at	TEXT	Timestamp of change

MODERATION	Moderation_id	INTEGER	PK
MODERATION	Submission_id	INTEGER	FK to SUBMISSION
MODERATION	decision	TEXT	Approved or returned
MODERATION	comment	TEXT	Moderation comment
MODERATION	Reviewd_at	TEXT	Timestamp of review

The dictionary is a proposed design. The final schema should be adjusted to match the implemented class diagram and the approved requirements.